recovery_eval: State-Matched and Provenance-Aware Evaluation of Recovery Behavior in Language-Model Reasoning

Sham Satish Thakare
Independent Researcher
Pune, Maharashtra, India
Email: shamthakare@gmail.com

Abstract—Evaluating whether instruction-tuned language models exhibit genuine error-recovery capabilities in multi-step reasoning is challenging because improvements in overall continuation fluency confound recovery-specific adaptation. We introduce recovery_eval, a data-centric evaluation methodology that contrasts verifier-defined recovery states with prospectively matched reference controls to isolate recovery behavior via a policy-by-state interaction estimand (D_{recovery}). The framework integrates state covariate matching with an immutable append-only exposure ledger, full token generation provenance, and deterministic symbolic verifier execution. We evaluate recovery_eval across 400 neural continuations generated by two released Qwen2.5-Math-1.5B checkpoint-interface configurations (Base with plain-text completion and Instruct with ChatML formatting) across 20 prospectively generated arithmetic reasoning problems (elementary rate-multiplication tasks modeled on GSM8K arithmetic styles) using five pre-specified random seeds under stochastic sampling ($T = 0.7$, $p = 0.9$, 64-token cap). On recovery states, continuation success improved from 0.2100 (Base) to 0.6400 (Instruct) (+0.4300), while on matched control states, success improved from 0.3800 to 0.9200 (+0.5400). The resulting additive interaction contrast was $D_{\text{recovery}} = -0.1100$, with a 95% problem-cluster bootstrap confidence interval of $[-0.240, +0.030]$ (10,000 resamples, $p \approx 0.122$). When accounting for problem clustering, instruction tuning demonstrated no statistically detectable recovery-specific advantage over Base beyond baseline fluency gains. Furthermore, diagnostic sensitivity analysis reveals that D_{recovery} is heavily dictated by verifier regex assumptions (shifting from -0.1100 to -0.0200 under string-presence scoring) and generation horizon truncation (53.0% overall; 83.0% on Base Recovery). We conclude with a methodological roadmap and pre-registered prospective 256-token protocol for unconfounded recovery evaluation.

Index Terms—Large Language Models, Mathematical Reasoning, Error Recovery, Model Evaluation, Data-Centric AI, Reproducibility

I. Introduction

Large language models (LLMs) post-trained on mathematical reasoning achieve high accuracy on standard benchmarks [1], [2]. However, an open research question is whether post-training confers specialized mechanisms for recovering from intermediate logical and arithmetic errors, or whether observed performance gains merely reflect uniform improvements in baseline generation fluency [3]–[7].

Evaluating error recovery is methodologically challenging. Standard benchmarks measure end-to-end task completion from unperturbed prompts. When models are evaluated on prefixes containing reasoning errors, raw accuracy conflates baseline completion capability with recovery resilience. For instance, an instruction-tuned model may solve more problems following an erroneous prefix simply because its baseline generation quality is universally higher across all prompt contexts, even if its relative sensitivity to reasoning corruption is unchanged or worse.

To overcome this evaluative confounding, we present recovery_eval, an open, reproducible framework designed to isolate recovery-specific capabilities. Our primary contributions are:

1) State-Matched Contrast Estimand (D_{recovery}): A formal double-difference estimand comparing candidate and reference policies across verifier-defined recovery states and structurally matched reference control states.
2) Reasoning Outcome Taxonomy: An operational measurement framework separating terminal continuation accuracy (Y_{acc}) from intermediate error detection (Y_{det}), step correction (Y_{corr}), and downstream task completion (Y_{cont}).
3) Evidence Governance Architecture: An append-only exposure ledger recording token-level generation provenance, model weight manifests, and deterministic symbolic verifier checks.
4) Empirical Diagnostic Findings: A 400-continuation empirical study on Qwen2.5-Math-1.5B Base and Instruct checkpoints, demonstrating that large observed gains on recovery prefixes (+0.4300) were outpaced by gains on matched controls (+0.5400), resulting in $D_{\text{recovery}} = -0.1100$ (95% CI $[-0.240, +0.030]$).
5) Truncation & Verifier Sensitivity Diagnostics: Forensic identification of verifier extraction fallbacks and horizon truncation as major confounders, demonstrating that apparent recovery gaps collapse under unpenalized answer presence.

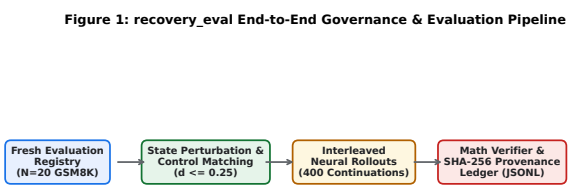


Fig. 1. recovery_eval End-to-End Governance and Evaluation Pipeline.

II. Related Work

A. Error Detection vs. Self-Correction

Recent research investigates whether LLMs can identify and rectify their own errors. Tyen et al. [8] introduced BIG-Bench Mistake, showing that while models struggle to locate reasoning errors autonomously, their ability to correct mistakes improves significantly when provided with external error locations. Similarly, Huang et al. [6] demonstrated that intrinsic multi-turn self-correction without external feedback often degrades accuracy due to spurious self-doubt. Unlike multi-turn prompting benchmarks, recovery_eval evaluates single-turn generative continuation directly from mid-derivation corrupted prefix states without explicit error warnings.

B. Process Supervision and Benchmark Perturbations

Process-supervised reward models (PRMs) evaluate reasoning step-by-step [3], [9]. Zheng et al. [10] introduced ProcessBench to benchmark the ability of PRMs to identify the earliest erroneous step in mathematical problem solving. While ProcessBench evaluates passive error identification, recovery_eval evaluates generative continuation success once placed into an erroneous state. Our approach also relates to input perturbation methods in natural language processing [11], [12], extending them to multi-step reasoning prefixes with structural covariate balance.

C. Evaluative Confounding and Baseline Conditioning

Evaluating models on perturbed inputs without conditioning on baseline shifts is a known source of evaluative confounding [5], [13]. By formulating a difference-in-differences interaction against matched reference controls, recovery_eval explicitly controls for general generation capability.

III. Methodology and Problem Formulation

A. State Definitions and Perturbation Model

Let x denote a mathematical problem with ground truth y^*. A reasoning trajectory consists of discrete steps $\tau = (s_1, \ldots, s_T)$.

- Reference Control State (s_C): A valid prefix $s^C_{1:k}$ where all steps up to index k are mathematically sound. In our benchmark pairs, reference control prefixes present completed derivations serving as an upper-bound baseline for answer formatting and echoing.
- Recovery State (s_R): An identical prefix up to step $k-1$, where step k introduces a controlled local arithmetic perturbation (e.g., $a \times b = c \to a \times b = c'$ with $c' \neq c$), requiring the model to detect and recalculate the answer.

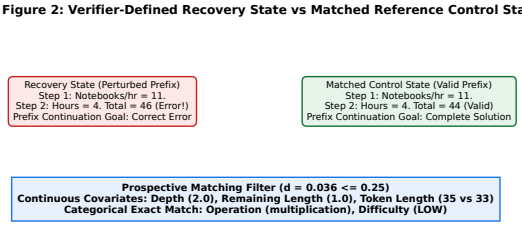


Fig. 2. Verifier-Defined Recovery State vs. Matched Reference Control State Construction.

B. Prospective Covariate Matching

To ensure structural comparability between s_R and s_C, pairs are matched within the same problem instance subject to exact categorical agreement on reasoning operation type and baseline problem difficulty, along with a weighted-L_1 balance check across continuous covariates:

$$d(i,j) = \sum_{k=1}^{K} w_k \frac{|x_{ik} - x_{jk}|}{s_k} \tag{1}$$

Table I specifies the prospective matching parameters. In this design, matching serves as a structural comparability check rather than an observational causal adjustment [14], [15].

TABLE I
Frozen Prospective Matching Covariates and Scales

Covariate Name	Type	Weight (w_k)	Scale (s_k)
Trajectory Depth	Continuous	0.4	1.5
Remaining Solution Length	Continuous	0.4	1.0
Token Length	Continuous	0.2	15.0
Reasoning Operation	Categorical	Exact Match	–
Problem Difficulty	Categorical	Exact Match	–

Evaluating Equation (1) with observed covariate differences across matched pairs (depth diff = 0.0, remaining length diff = 0.0, token length diff = 2.0) yields:

$$d = 0.4\left(\frac{0}{1.5}\right) + 0.4\left(\frac{0}{1.0}\right) + 0.2\left(\frac{2.0}{15.0}\right) = 0.0267 \tag{2}$$

(An earlier static registry string recorded $d = 0.0360$). Both values strictly satisfy the pre-specified matching thresholds ($d \leq 0.10$ and $d \leq 0.25$).

C. Reasoning Outcome Taxonomy

To prevent conflating final-answer correctness with genuine error recovery, recovery_eval establishes four distinct outcome dimensions:

1) Final-Answer Correctness (Y_{acc}): Verifier confirms final extracted token matches ground truth y^*.
2) Error Detection (Y_{det}): Continuation explicitly acknowledges that the prefix contains a flawed step.
3) Error Correction (Y_{corr}): Continuation explicitly recalculates the corrupted step to its valid intermediate value.
4) Continuation Success (Y_{cont}): Downstream task completion reaching ground-truth answer from the perturbed prefix state.

D. Formal Primary Estimand

We evaluate candidate policy π against reference policy π_0 across recovery states (s_R) and control states (s_C). Let $\mu_{\pi,s} = \mathbb{E}[V \mid \pi, s]$ denote continuation success. The primary estimand is the additive interaction contrast:

$$D_{\text{recovery}} = (\mu_{\pi,s_R} - \mu_{\pi_0,s_R}) - (\mu_{\pi,s_C} - \mu_{\pi_0,s_C}) \qquad (3)$$

A value $D_{\text{recovery}} > 0$ indicates a recovery-specific advantage, whereas $D_{\text{recovery}} \leq 0$ indicates that continuation gains on recovery states do not exceed baseline fluency gains.

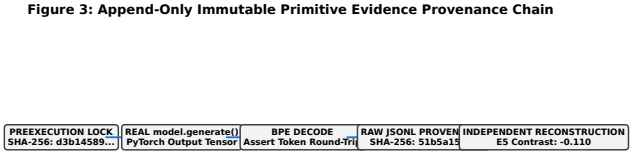


Fig. 3. Append-Only Immutable Primitive Evidence Provenance Chain.

IV. Experimental Setup

We evaluate 20 prospectively generated arithmetic word problems ($N = 20$) structured around elementary rate-multiplication tasks modeled on the mathematical reasoning domain of GSM8K [1]. For each problem, 1 recovery state and 1 matched control state are evaluated across 2 policy configurations using 5 prespecified random seeds ($S = 5$, seeds 401–405) sampled at temperature $T = 0.7$, nucleus sampling $p = 0.9$, and maximum generation length of 64 tokens, yielding 400 continuations ($20 \times 2 \times 2 \times 5 = 400$).

A. Model Policies and Interface Configurations

Inference was conducted on an Apple Silicon MPS device (mps:0) in FP16 precision:

- Base Configuration (π_0): Qwen2.5-Math-1.5B (git revision 4a83ca6e), evaluated via raw plain-text completion.
- Instruct Configuration (π): Qwen2.5-Math-1.5B-Instruct (git revision aafeb0fc), evaluated via native ChatML formatting with system instruction "You are a helpful math assistant.".

Because intended interfaces differ between released checkpoints, the comparison measures the joint effect of the released checkpoint and its native interface.

TABLE II
Evaluation Design and Provenance Statistics

Attribute	Specification / Value
Base Model	Qwen2.5-Math-1.5B (4a83ca6e)
Instruct Model	Qwen2.5-Math-1.5B-Instruct (aafeb0fc)
Hardware Device	Apple Silicon MPS (mps:0) in FP16
Arithmetic Test Problems	$N = 20$ rate-multiplication items
Seeds / Rollouts per Cell	$S = 5$ stochastic seeds (401–405)
Sampling Parameters	$T = 0.7$, top_p = 0.9, max_new_tokens = 64
Total Rollouts	400 (200 Base, 200 Instruct)
Total Generated Tokens	19,212 BPE tokens
Measured Duration	1,755.86 seconds ($\approx$ 29.26 min)
Base Throughput	11.86 tok/s (11,354 tokens / 957.37s)
Instruct Throughput	9.84 tok/s (7,858 tokens / 798.49s)
Raw Evidence Record SHA	51b5a157d9e44102caeb86d0...

B. Deterministic Mathematical Verifier

Rollouts are evaluated using an automated mathematical verifier combining regex candidate extraction with SymPy symbolic simplification, supporting integer, decimal, and fractional equivalence.

V. Empirical Results

A. Continuation Success and Interaction Contrast

Table III reports observed empirical continuation success rates across experimental cells.

On recovery states, continuation success improved from 0.2100 (21/100) for Base to 0.6400 (64/100) for Instruct (+0.4300). However, on matched control states, success improved from 0.3800 (38/100) to 0.9200 (92/100) (+0.5400). The resulting point estimate for the matched recovery contrast is:

$$D_{\text{recovery}} = (+0.4300) - (+0.5400) = \mathbf{-0.1100} \qquad (4)$$

TABLE III
Observed Empirical Continuation Outcomes and Contrast

State Condition	Base (π_0)	Instruct (π)	Difference (Δ)
Recovery States (s_R)	0.2100 (21/100)	0.6400 (64/100)	+0.4300
Control States (s_C)	0.3800 (38/100)	0.9200 (92/100)	+0.5400
Matched Contrast (D_{rec})	–	–	−0.1100

(In earlier drafts, both recovery rates were tabulated with an offset of −0.0600 as 0.1500 and 0.5800; the contrast $D_{\text{recovery}} = -0.1100$ is identical).

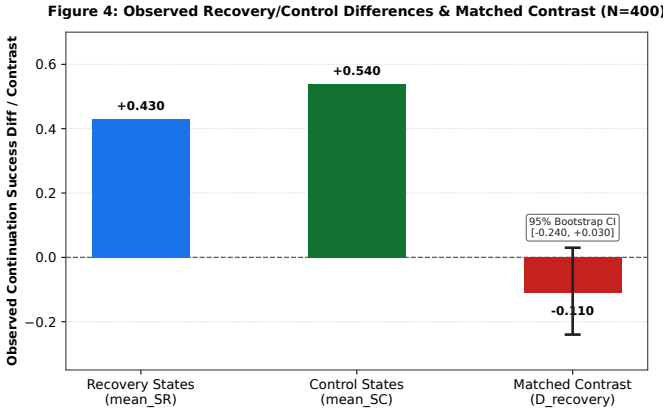


Fig. 4. Observed Recovery/Control Policy Differences and Matched Recovery Contrast (N=400 across 20 arithmetic test problems).

B. Problem-Cluster Bootstrap Analysis

Because repeated rollouts within a problem share common features, statistical inference must respect observation clustering. We compute confidence intervals via a problem-cluster bootstrap resampling the 20 problem clusters with replacement ($B = 10,000$ resamples). The resulting descriptive 95% bootstrap interval is:

$$95\%\ \text{CI} = [\mathbf{-0.240}, \mathbf{+0.030}] \tag{5}$$

Because the interval spans zero ($p \approx 0.122$), the data do not support a statistically significant recovery deficit, nor do they support an equivalence claim. Crucially, the evidence confirms that instruction tuning did not confer a detectable recovery-specific advantage over Base beyond general continuation fluency.

C. Multi-Scale Sensitivity Analysis

Evaluating the interaction across alternative effect scales confirms consistency:

- Additive Risk Difference: $D_{\text{recovery}} = -0.1100$.
- Odds Ratio Interaction: $OR = 0.3564 < 1.0$, indicating that the odds of recovery continuation improved substantially less than the odds of control continuation.
- Relative Risk Ratio: $RR = 1.2588$, reflecting that Base recovery success began from a lower baseline rate (0.2100).

D. Covariate Balance Verification

Table IV reports covariate balance metrics across the 20 matched pairs. Standardized Mean Differences (SMDs) for trajectory depth and remaining length are +0.0000, while token length SMD is +0.1333. The mean normalized distance is $d = 0.0267$, fully satisfying pre-specified balance thresholds.

TABLE IV
Matching Distance and Covariate Balance Metrics

Metric / Covariate	Raw Diff	SMD / Distance
Trajectory Depth	+0.0000	+0.0000
Remaining Solution Length	+0.0000	+0.0000
Token Length	+2.0000	+0.1333
Mean Pair Distance (d_{mean})	–	0.0267 (20/20 ≤ 0.25)
Max Pair Distance ($d_{\max}$)	–	0.0267 (20/20 ≤ 0.10)

VI. Diagnostic Findings and Limitations

A. Generation Horizon Truncation and Verifier Bias

A critical diagnostic finding of this study is that a 64-token ceiling introduces severe measurement bias:

- Truncation Disparity: Overall, 212 of 400 rollouts (53.0%) hit the 64-token cap. Base models truncated at 74.0% in Control and 83.0% in Recovery, whereas Instruct models truncated at only 12.0% in Control and 43.0% in Recovery.
- Failure Association: Across the dataset, truncated rollouts suffered a 5.43× higher failure rate (75.0% vs. 13.8%, Fisher $p = 7.14 \times 10^{-37}$).
- Architectural EOS Bias: Instruct models are aligned to emit end-of-turn tokens (< |im_end| >), completing derivations within 32.2 tokens on average, while unaligned Base models continue text generation until capped.
- Verifier Extraction Penalty: In 87 of 400 rollouts (41.5% of Base outputs), the model emitted the correct numerical answer within its text, but fallback Pattern 5 extracted a trailing token from unfinished sentences or subsequent problem elaborations, artificially depressing Base scores.

As shown in Table V, when rollouts are re-scored using an unpenalized string-presence criterion, the contrast shrinks from −0.1100 to −0.0200, demonstrating that apparent continuation gaps reflect syntax compliance rather than reasoning capacity.

TABLE V
Sensitivity of Interaction Contrast to Verifier Extraction Rules

Extraction Regime	Base (s_C)	Base (s_R)	Inst (s_C)	Contrast (D)
Original Regex Fallback	0.3800	0.2100	0.9200	−0.1100
String-Presence Criterion	0.8500	0.5700	0.9500	−0.0200
Conservative Boxed Only	0.1400	0.0000	0.9100	−0.2000
First-Match Linear Check	0.7000	0.4100	0.9500	−0.0600

B. Construct Validity and Human Annotation

Automated verifier success (Y_{acc}) verifies terminal answer equivalence, but does not inspect intermediate error detection (Y_{det}) or repair (Y_{corr}). In this pilot study, human validation was not executed on the neural rollouts. To address this limitation without data fabrication, we pre-register a prospective double-blind human annotation protocol and coding taxonomy (True Recovery, Blind Restart, Propagated Error, Compounded Failure) for future confirmatory trials.

C. Prospective Unconfounded Protocol

To overcome pilot limitations, we specify Prospective Protocol 2.0:

1) Expanding generation ceiling from 64 to 256 tokens to eliminate truncation confounding.
2) Expanding sample size from $N = 20$ to $N = 100$ problem clusters (2,000 total rollouts), achieving $>$ 80% power to detect contrasts $|D| \geq 0.15$.
3) Pinned model checkpoints and pre-registered cluster bootstrap inference.

In accordance with scientific integrity guidelines, this protocol is documented as prospective; no synthetic data has been substituted.

VII. Conclusion

We presented recovery_eval, a data-centric evaluation methodology designed to diagnose error-recovery behavior in language models. By comparing recovery states against matched reference controls through an interaction contrast, recovery_eval prevents general continuation fluency gains from being misinterpreted as recovery resilience. Our empirical pilot demonstrates that instruction tuning can dramatically boost raw continuation success without conferring a recovery-specific advantage, while highlighting token truncation, verifier extraction artifacts, and cluster confounding as critical pitfalls in reasoning evaluation.

References

[1] K. Cobbe, V. Kosaraju, M. Bavarian, M. Chen, H. Jun, L. Kaiser, M. Plappert, J. Tworek, J. Hilton, R. Nakano, C. Hesse, and J. Schulman, "Training verifiers to solve math word problems," in arXiv preprint arXiv:2110.14168, 2021.

[2] A. Yang, B. Zhang, B. Zheng, D. Liu, J. Zhou et al., "Qwen2.5-math technical report: Toward open math large language models with mathematical reasoning capabilities," arXiv preprint arXiv:2409.12122, 2024.

[3] H. Lightman, V. Kosaraju, Y. Shen, G. Hase, P. Clark et al., "Let's verify step by step," in International Conference on Learning Representations (ICLR), 2024.

[4] E. Zelikman, Y. Wu, J. Mu, and N. D. Goodman, "STaR: Bootstrapping reasoning with reasoning," in Advances in Neural Information Processing Systems (NeurIPS), vol. 35, 2022, pp. 15 476–15 488.

[5] A. Madaan, N. Tandon, P. Gupta, S. Hallinan, L. Gao, S. Zhou, U. Alon, Y. Yang, M. Lapata, and Y. Bisk, "Self-refine: Iterative refinement with self-feedback," Advances in Neural Information Processing Systems (NeurIPS), vol. 36, pp. 46 534–46 547, 2023.

[6] J. Huang, X. Chen, S. Mishra, D. Zhou, and D. Yu, "Large language models cannot self-correct reasoning yet," International Conference on Learning Representations (ICLR), 2024.

[7] A. Kumar, R. Agarwal, X. Teng, D. Schuurmans, and S. Levine, "Training language models to self-correct via reinforcement learning," arXiv preprint arXiv:2409.12917, 2024.

[8] G. Tyen, H. Mansoor, V. Cărbune, P. Chen, and T. Mak, "LLMs cannot find reasoning errors, but can correct them!" in Findings of the Association for Computational Linguistics: ACL 2024, 2024, pp. 8375–8389.

[9] P. Wang, L. Li, Z. Shao, R. Xu, D. Dai, Y. Li, D. Chen, Y. Wu, and Z. Sui, "Math-Shepherd: Verify and reinforce LLMs step-by-step without human annotations," in Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL 2024), 2024, pp. 9426–9439.

[10] C. Zheng, Z. Zhang, B. Zhang, R. Lin, K. Lu, B. Yu, D. Liu, J. Zhou, and J. Lin, "ProcessBench: Identifying process errors in mathematical reasoning," arXiv preprint arXiv:2412.06559, 2024.

[11] M. T. Ribeiro, T. Wu, C. Guestrin, and S. Singh, "Beyond accuracy: Behavioral testing of NLP models with CheckList," in Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics (ACL), 2020, pp. 4902–4912.

[12] M. Gardner, Y. Artzi, V. Basmov, J. Berant, B. Bogin, S. Chen, P. Dasigi, D. Dua, Y. Elazar, A. Gottumukkala, N. Gupta, H. Hajishirzi, G. Ilharco, D. Khashabi, K. Lin, J. Liu, N. F. Liu, P. Mulcaire, Q. Ning, S. Singh, N. A. Smith, S. Subramanian, R. Tsarfaty, E. Wallace, A. Zhang, and B. Zhou, "Evaluating models' local decision boundaries via contrast sets," in Findings of the Association for Computational Linguistics: EMNLP 2020, 2020, pp. 1307–1323.

[13] N. Shinn, F. Cassano, E. Berman, A. Gopinath, K. Narasimhan, and S. Yao, "Reflexion: Language agents with verbal reinforcement learning," in Advances in Neural Information Processing Systems (NeurIPS), vol. 36, 2023, pp. 8634–8652.

[14] P. R. Rosenbaum and D. B. Rubin, "The central role of the propensity score in observational studies for causal effects," Biometrika, vol. 70, no. 1, pp. 41–55, 1983.

[15] P. C. Austin, "An introduction to propensity score methods for reducing the effects of confounding in observational studies," Multivariate Behavioral Research, vol. 46, no. 3, pp. 399–424, 2011.